



API Standart de Java



En la asignatura, **API Standard** significa *la parte clásica de Java*: `Object`, `Thread`, `synchronized`, `wait/notify`. Todo lo que NO es `java.util.concurrent` se considera estándar.

Explicación

Es importantísimo distinguir entre:

- **Modelo teórico** (monitores, semáforos, RCC...)
- **API Standard** (lo que se puede simular)
- **API de alto nivel** (lo que se puede implementar directamente)

Muchísimas preguntas de examen preguntan **qué se puede y qué NO se puede implementar** según el nivel de API.

Peculiaridades

Dentro de esta versión de la API NO TENEMOS NINGUNA IMPLEMENTACIÓN DIRECTA de las primitivas de control de la concurrencia que se estudian. Tan solo podemos simular sus comportamientos.

- ❌ semáforos
- ❌ variables de condición
- ❌ cerrojos explícitos
- ❌ monitores "de verdad" (con múltiples colas)

Lo que tenemos es:

- ✓ **exclusión mutua a nivel de objeto** mediante `synchronized`

- ✓ **una única cola de espera** por objeto (el *wait-set*)
- ✓ **wait(), notify(), notifyAll()**
- ✓ creación de hilos con `Thread` o `Runnable`

Exclusión mutua

En este nivel, **la exclusión mutua es siempre a nivel de objeto**, porque:

- **todo objeto Java contiene un cerrojo interno.**
 - Esto se debe a que la clase **Object** es la raíz en la jerarquía de clases de Java. Todos los objetos, implementan los métodos de esta clase. Y la clase **Object** nos provee de métodos y mecanismos para lograr exclusión mutua y sincronización.
- **solo podemos usar ese cerrojo**
- **no podemos crear cerrojos explícitos** (eso sería API de alto nivel)

Se usa `synchronized` de dos formas:

Bloque sincronizado

Este bloque de código puede utilizar como objeto para usar el cerrojo un objeto cualquiera que pasamos como argumento, o `this`.

```
synchronized (obj) {
    // zona crítica
}
```

Ejemplo real:

```
public class Contador {
    private int valor = 0;

    public void incrementar() {
        synchronized (this) { // cerrojo del propio objeto Contador
            valor++;
        }
    }
}
```

```
}  
}
```

Método sincronizado

Este método, al ser un método de clase, obligatoriamente utilizará `this` como cerrojo.

```
public synchronized void incrementar() {  
    valor++;  
}
```

Esto es exactamente igual que:

```
public void incrementar() {  
    synchronized (this) {  
        valor++;  
    }  
}
```

Sincronización

La sincronización viene dada por **los métodos de Object**:

- `wait()` . Al hacer `wait()` , el hilo libera el cerrojo y pasa al *wait-set* del objeto. Cuando otro hilo hace `notify` / `notifyAll` , los hilos despertados pasan a "listos" pero deben volver a competir por el cerrojo.
- `notify()` . Despierta a un hilo del *wait-set* para que pueda competir por el cerrojo. Como no podemos establecer prioridades, el hilo que se despierta será uno arbitrario de los que esté esperando, peor no podemos asegurar cuál. Por este motivo, en implementación siempre se nos recomienda utilizar directamente `notifyAll` .
- `notifyAll()` . Despierta a todos los hilos que estaban esperando.

Estos métodos *siempre* trabajan sobre el **wait-set del objeto cuyo cerrojo se está usando**.

Reglas a tener en cuenta para usar estos métodos

1. Solo se pueden usar dentro de `synchronized`

Porque necesitan saber **qué cerrojo liberar** y **qué wait-set** manipular.

Si no lo haces, Java lanza `IllegalMonitorStateException`.

2. `wait()` libera el cerrojo y bloquea el hilo

El hilo pasa al **wait-set** del objeto.

Cuando otro hilo hace `notify` o `notifyAll`, el hilo despertado:

- NO entra directamente al monitor
 - pasa a *listo*
 - **compite de nuevo por el cerrojo**
 - cuando entra → reevaluamos la condición (por eso usamos `while`)
-

3. `notify()` despierta a *un hilo cualquiera*

No hay prioridades, no hay orden.

No podemos garantizar FIFO ni equidad.

4. `notifyAll()` despierta a *todos*

Es más seguro, pero más costoso:

todos se despiertan, reevalúan condición, la mayoría vuelven a dormir.

5. Si el wait-set está vacío... la señal se pierde

`notify()` y `notifyAll()` **no quedan almacenados**.

Como toda clase implementa los métodos de `Object`, todo objeto que creamos tendrá estos métodos.

Creación de Hilos

La Máquina Virtual Java (MVJ) permite que una aplicación tenga múltiples hilos ejecutándose concurrentemente.

Estos hilos son mapeados a hilos del sistema operativo, por lo que consumen recursos.

Tenemos 2 formas de definir hilos en Java:

- Heredando de **Thread**.

```
class MiHilo extends Thread {  
    public void run() {  
        // código del hilo  
    }  
}  
  
MiHilo h = new MiHilo();  
h.start();
```

- Implementando la interfaz **Runnable**. Esta es la opción preferible porque Java no tiene herencia múltiple.

```
class MiTarea implements Runnable {  
    @Override  
    public void run() {  
        // código del hilo  
    }  
}  
  
Thread t = new Thread(new MiTarea());  
t.start();
```